

Oluwabukunmi Odukoya

bukunmiodukoya@gmail.com
+234 707 339 4962
kunmisportfolio.xyz
linkedin.com/in/oluwabukunmi-odukoya

AxisCare

June 2026

Dear Hiring Team,

The most honest answer I can give to your first question is: I built a crypto prediction market, and AI was deeply woven into how I got it done.

The product — Predict Market — lets users trade binary yes/no positions on crypto outcomes. In-app wallets, a referral system, real-time balance tracking, market settlement via cron jobs. Full TypeScript, Next.js, Supabase with Row-Level Security. The kind of system that has a dozen places where the wrong decision buries you quietly.

I used AI tools throughout — primarily Claude — not as a code generator I pasted from, but as a technical collaborator I interrogated. Where it genuinely shone: Supabase's RLS policy syntax is dense and the error messages are almost deliberately unhelpful. AI dramatically shortened the loop between "something is wrong with my policy" and "here's exactly why." Same with cron-job settlement logic — I described the edge cases I was worried about (race conditions, duplicate settlements, clock drift) and the back-and-forth forced me to articulate assumptions I hadn't written down. That's genuinely useful.

Where it fell short: anything that required understanding the full system at once. When I asked for help with wallet balance consistency across concurrent transactions, I got technically correct answers that missed the actual constraint — that Supabase's realtime subscriptions and RLS interact in a specific way in my setup. The model didn't know what it didn't know about my architecture, and neither did I until I'd read the output carefully, pushed back, and debugged it myself. It confidently suggested an approach I had to throw away.

What I learned about my role: the work didn't get smaller — it shifted. Less time on syntax and boilerplate, more time on judgment. Deciding which suggestions to trust, which to test, which to bin. You still have to hold the full system in your head. You still have to know what "correct" looks like to recognise when the tool is wrong. If anything, AI surfaces how much of good engineering is taste and context that doesn't fit in a prompt.

That's why AxisCare is where I want to be right now. You're not dabbling in AI — you just shipped AxisCare Intelligence: scheduling recommendations, care analytics, an AI assistant, all embedded into live workflows, HIPAA-ready, built on AWS Bedrock. That's a team that understands the difference between bolting on a chatbot and actually integrating intelligence into a domain where the stakes are real people's care. Home care operations aren't forgiving. Bad scheduling logic doesn't just cause bugs — it leaves someone without a caregiver. That constraint is exactly the kind of thing that makes backend engineering interesting, and AI augmentation meaningful rather than cosmetic. I want to be building in that environment, with engineers who are thinking carefully about what these tools are actually good for.

Sincerely,

Oluwabukunmi Odukoya

